

agent-guardrail

A Deterministic Policy Firewall for AI Agent Tool Calls

rudimentall1 · August 2026 · v1.0.2

Abstract

Most guardrail approaches for AI agents rely on a learned or heuristic risk score — a number an operator is asked to trust without being able to read why it moved. agent-guardrail takes the opposite approach: every decision is the output of explicit, human-authored YAML rules, evaluated deterministically, so a blocked action can always be traced to the exact rule that blocked it. This paper describes the policy model, the audit and confirmation layers built around it, and what remains a deliberate design boundary rather than an oversight.

1. Motivation

As agents are given tool access — shell execution, wallet operations, arbitrary HTTP calls — the operator's real question is rarely "how risky is this on a 0–100 scale" and almost always "did this match a rule I actually wrote, and can I see which one." A risk score is a summary an operator has to trust; a matched rule is a fact an operator can check.

agent-guardrail is built around that distinction. It intercepts a proposed tool call, evaluates it against a policy file the operator controls, and returns ALLOW, WARN and require confirmation, or BLOCK — with the specific rule and reason attached to every outcome.

2. Policy Model

A policy is a YAML document mapping tool names (or patterns) to argument-level rules: numeric caps, string/regex pattern matches against arguments, and per-tool default dispositions. Rules compose in a fixed, documented precedence order rather than a weighted blend, so the same policy always produces the same decision for the same input — a property a scored system cannot offer without also freezing its model.

```
1) Small, known-pattern transfer:
  [confirmation required] ["Tool 'wallet.transfer' always requires human confirmation"]
  -> executing transfer: 2 to 0xabc

2) Oversized transfer from a brand-new agent:
  blocked as expected: ["amount=9999.0 exceeds cap 5 for 'wallet.transfer' (unknown
  agent)"]

3) Destructive shell command:
  blocked as expected: ['Argument contains a known destructive or self-replicating
  shell pattern']
```

Live output from examples/example_agent_usage.py — three real decisions, three real reasons.

3. Verifiable Audit Trail

Every decision is optionally signed as an OAA (Open Agent Attestation) token, the same open, independently published Ed25519/JWT format used by this author's agentic-wallet-guardian-v3 project. A third party holding the public key can verify a decision was genuinely produced by this engine, offline, without trusting whichever server is presenting the record.

decision: BLOCK

OAA token:

eyJhbGciOiJIJZERTQSI6InR5cCI6IkpXVCJ9.eyJpc3Mi...vodRYGs_vyNt

verified by a third party:

```
decision: BLOCK
action: tool_call:wallet.transfer
reason: amount=9999.0 exceeds cap 5 for 'wallet.transfer' (unknown agent)
```

4. Secrets-Aware Logging

A local audit log records every decision for later review — which puts the log's own contents in direct tension with anything sensitive an agent's arguments might carry. Redaction is on by default: values under a key that looks sensitive (password, api_key, authorization, private_key, and their common header-style spellings) are replaced before persisting, as are two high-confidence value shapes — PEM private key blocks and JWT-shaped strings — regardless of what the key is named.

A general-purpose "looks like a random secret" entropy heuristic was deliberately left out: it would flag ordinary UUIDs and hashes as often as real secrets, and a security tool that cries wolf constantly trains its operator to stop reading its warnings.

5. Distribution

agent-guardrail ships as an installable Python package and an MCP server, and is listed in the official Model Context Protocol Registry — installable directly by any MCP-compatible agent runtime without a manual integration step.

```
pip install guardrail-mcp
```

6. Honest Status

59 tests pass across the suite as of this writing, including the redaction module documented above. The following are documented design tradeoffs, not gaps expected to close automatically:

- Single-process SQLite by default — sufficient for one agent process; multi-replica deployments should point at shared storage or swap in a real database.
- The default policy is a reasonable starting point, not a complete threat model — it is expected to be extended per deployment.
- The confirmation web UI has no authentication; it binds to 127.0.0.1 by design and is not intended to be network-exposed.

7. Conclusion

agent-guardrail's bet is that legibility beats sophistication for a security control: an operator who can point at the exact line of YAML that blocked an action trusts the system more than one asked to accept a score. Source and tests: github.com/rudimentall1/agent-guardrail